VECTOR

Integrating EEP with Infineon SPI Driver using Safety Feature

Version 1.0.0
2017-01-23
Application Note AN-ISC-8-1200

Author Michael Goß

Abstract This Application Note gives instructions how to implement a wrapper for Infineon's SPI driver in case its SpiSafetyEnable switch is enabled.
If SpiSafetyEnable is activated, the Infineon SPI driver checks if the correct safety marker is available in any buffer parameters which are passed to SPI. Due to the fact that MICROSAR EEP driver does not add any markers to buffers, a wrapper needs to be implemented by the user, which takes care of adding safety markers before any buffers are passed to the SPI driver.
This document is a guide for implementing a wrapper according to the safety feature realized in Infineon's SPI driver.

Table of Contents

1 Overview

As an introduction chapter 2 describes the feature in Infineon's SPI driver, which makes it necessary to implement a SPI wrapper. Afterwards the implementation of this wrapper will be discussed in chapter 3.

2 Infineon SPI Safety Feature

Safety feature of Infineon's SPI driver can be enabled and disabled via a configuration switch in SPI configuration called **SpiSafetyEnable**. In case this feature is enabled, the buffer parameters which are passed from other components to SPI are checked for valid safety markers, so called SpiSafetyBufferMarkers.

SpiSafetyBufferMarkers are usually configured uniquely for each SPI channel and used as end buffer marker for each channel. That means that the SpiSafetyBufferMarker, which corresponds to a channel, is appended to the end of the buffer before the buffer is set up for the specified channel e.g. in `Spi_SetupEB` service.

For example, EEP driver calls `Spi_SetupEB` service at some point with a valid `SrcDataBufferPtr` and a `DesDataBufferPtr` equal to `NULL_PTR`, which would be the case in a write job. Additionally EEP driver passes the SPI channel and the length of the buffer to `Spi_SetupEB` service. Let the length parameter be 2 in this example, therefore `SrcDataBufferPtr[0]` and `SrcDataBufferPtr[1]` contain data. `Spi_SetupEB` will then check if `SrcDataBufferPtr[2]` will contain the SpiSafetyBufferMarker which corresponds to the specified SPI channel.

Caution
It must be assured that the buffers are big enough to hold the additional marker information.

The following code snippet shall again show the situation which is described above:

Example

```
uint8 SrcDataBufferPtr[3];
uint8 * DesDataBufferPtr = NULL_PTR;

SrcDataBufferPtr[0] = data1;
SrcDataBufferPtr[1] = data2;
SrcDataBufferPtr[2] = SPI_CHANNEL1_SAFETY_MARKER; /* marker must
correspond to channel parameter in Spi_SetupEB call */

Length = 2; /* Safety marker will be checked implicitly by Spi_SetupEB
service. Length parameter doesn't take the safety marker into
consideration */

Spi_SetupEB(Channel1, SrcDataBufferPtr, DesDataBufferPtr, Length);
```

`Spi_SetupEB` assumes that a safety marker is available at index **Length** of the passed buffer. Due to the fact that this position is out of bounds of the usual data contained in the buffer, the user has to ensure that the buffer is big enough to hold this additional marker value.

Due to AUTOSAR compliance, MICROSAR EEP driver does not provide a safety marker at the end of any buffers passed to SPI, thus it's necessary to implement a SPI wrapper which adds the marker byte corresponding to current SPI channel at the end of the buffer. After adding this marker, the wrapper calls the corresponding SPI service.

The following chapter explains all necessary steps to implement this SPI wrapper.

3 Wrapper Implementation

First section deals with the wrapper interface and the according EEP driver configuration, so that the wrapper is used by EEP driver instead of the original SPI driver. The following section describes the measures that have to be taken in order to copy the safety marker to the end of the buffer. Finally, the last section will explain what is necessary in case of read jobs.

3.1 Wrapper Interface

The EEP driver uses the following SPI services, which therefore need to be implemented by the SPI wrapper component:

- `Spi_SetupEB`
- `Spi_AsyncTransmit`
- `Spi_GetSequenceResult`

The wrapper needs to implement these services with the same signature but with different name, e.g.:

- `SpiWrapper_SetupEB`
- `SpiWrapper_AsyncTransmit`
- `SpiWrapper_GetSequenceResult`

From now on this Application Note will stick to these example names for the wrapper functions.

The SPI interface is configurable in EEP driver's configuration, thus there is a way to announce the wrapper's services instead of the original SPI services. For further information check the user manual of EEP driver.

The next section describes how a temporary buffer needs to be provided in the SPI wrapper so that the safety marker can be appended to the original data.

3.2 Provide a Temporary Buffer to Store the Safety Buffer Marker

In the function `SpiWrapper_SetupEB` it is necessary to copy the SpiSafetyBufferMarker related to current channel to the end of the buffer. The `SetupEB` service will always be called with either the `SrcDataBufferPtr` or `DesDataBufferPtr` or both of them being `NULL_PTR`. Thus it is easy to decide where the safety marker needs to be copied to.

Consequently, the **FIRST STEP** is to check both buffer pointers for `NULL_PTR` and continue with the one which is valid. Buffer pointers which are `NULL_PTR` can be ignored. The input buffer pointer which is **not** `NULL_PTR` will be called `InputBufferPtr` from now on.

Due to the fact that the user buffer, which is passed to the wrapper component, does not provide additional space for the SpiSafetyBufferMarker, it is necessary to create a local buffer in the wrapper component. This local buffer needs to be big enough to hold the maximum size of EEP driver's job plus the size of the SpiSafetyBufferMarker, which is assumed to be one byte for further considerations. The maximum size of an EEP job, a so called chunk size, can be determined by the following configuration parameters:

Caution
Maximum chunk size (MaxEepChunkSize) is the maximum value of following parameters:

- EepInitConfiguration/EepNormalReadBlockSize
- EepInitConfiguration/EepFastReadBlockSize
- EepInitConfiguration/EepNormalWriteBlockSize
- EepInitConfiguration/EepFastWriteBlockSize
- EepDeviceInformation/EepPageSize

Therefore the local buffer, which needs to be defined in the wrapper, has to be defined like:

Example

```
Spi_DataType SpiWrapper_TempBuffer[MaxEepChunkSize + 1];
/* One additional byte is necessary for the safety marker */
```

It is assumed that `Spi_DataType` is of datatype **unsigned char** (uint8)!

The **SECOND STEP** of adding the safety marker is copying the information referenced by the input buffer pointer to the local buffer of the wrapper:

Example

```
for (index = 0; index < Length; index++)
{
    SpiWrapper_TempBuffer[index] = inputBufferPtr[index];
}
SpiWrapper_TempBuffer[Length] = SPI_CHANNELX_SAFETY_MARKER;
/* Length is input parameter of SpiWrapper_SetupEB */
```

The value of the safety buffer marker depends on the Channel parameter, which is input of `SpiWrapper_SetupEB` function. The correct values can be taken from the SPI channel configurations and could probably be hard-coded in the wrapper component. Consequently, it's only necessary to check which channel is used in a call of `SpiWrapper_SetupEB` and copy the related safety marker value to the local buffer, as depicted in the example above.

The **THIRD STEP** (and las one) in `SpiWrapper_SetupEB` function is to call the corresponding SPI service `Spi_SetupEB` with the temporary buffer pointer instead of the original pointer, as shown in the following example. Depending on the original input buffer pointers, it is important to avoid mixing up the parameters: `NULL_PTR` stays `NULL_PTR` and valid pointer stays valid. See the example:

Example

```
Std_ReturnType retVal;
if ((SrcDataBufferPtr != NULL_PTR) && (DesDataBufferPtr == NULL_PTR))
{
      retVal = Spi_SetupEB(Channel, SpiWrapper_TempBuffer, NULL_PTR,
      Length);
}
else if((SrcDataBufferPtr == NULL_PTR) && (DesDataBufferPtr != NULL_PTR))
{
      retVal = Spi_SetupEB(Channel, NULL_PTR, SpiWrapper_TempBuffer,
      Length);
}
else
{
      retVal = Spi_SetupEB(Channel, NULL_PTR, NULL_PTR, Length);
}
return retVal;
```

3.3 Copy Buffer Upon End of a Read Job

For any write jobs (i.e. `SrcDataBufferPtr` != `NULL_PTR`) no more measures need to be taken into account in the SPI wrapper.

In contrary, read jobs (i.e. `DesDataBufferPtr` != `NULL_PTR`) do need to be processed with additional care. Since it is intended to retrieve data from lower layers within a read job, the content of `SpiWrapper_TempBuffer` needs to be copied back to the original `DesDataBufferPtr` after finishing the SPI sequence.

Therefore the SPI wrapper needs to store somehow the information that the current sequence is a “read sequence” (i.e. `DesDataBufferPtr` != `NULL_PTR`) and copy the data from `SpiWrapper_TempBuffer` to the original `DesDataBufferPtr` in `SpiWrapper_GetSequenceResult` function upon end of sequence communication. For this purpose it is also necessary to store the `DesDataBufferPtr` reference in order to know where the recently read data needs to be copied to.

Consequently the user needs to save the `DesDataBufferPtr` reference and `Length` parameter upon call of `SpiWrapper_SetupEB` function, if `DesDataBufferPtr` is valid (i.e. is not `NULL_PTR`).

Furthermore the `SpiWrapper_GetSequenceResult` function needs to take care of copying the data back to the original buffer, as depicted by the following example:

Example

```
/* In case of a READ SEQUENCE do the following */
Spi_SeqResultType spiResult = Spi_GetSequenceResult();
if (spiResult == SPI_SEQ_OK)
{
   /* Copy data of temp buffer to original buffer */
   for (index = 0; index < savedLength; index++)
   {
      savedDesDataBufferPtr[index] = SpiWrapper_TempBuffer[index];
   }
}
/* Return result to EEP */
return spiResult;

/* savedLength and savedDesDataBufferPtr are the original
parameters which were saved upon call of SpiWrapper_SetupEB
function */
```

4 Contacts

For a full list with all Vector locations and addresses worldwide, please visit http://vector.com/contact/.